ASEAN
GEOAI FUSION 2026

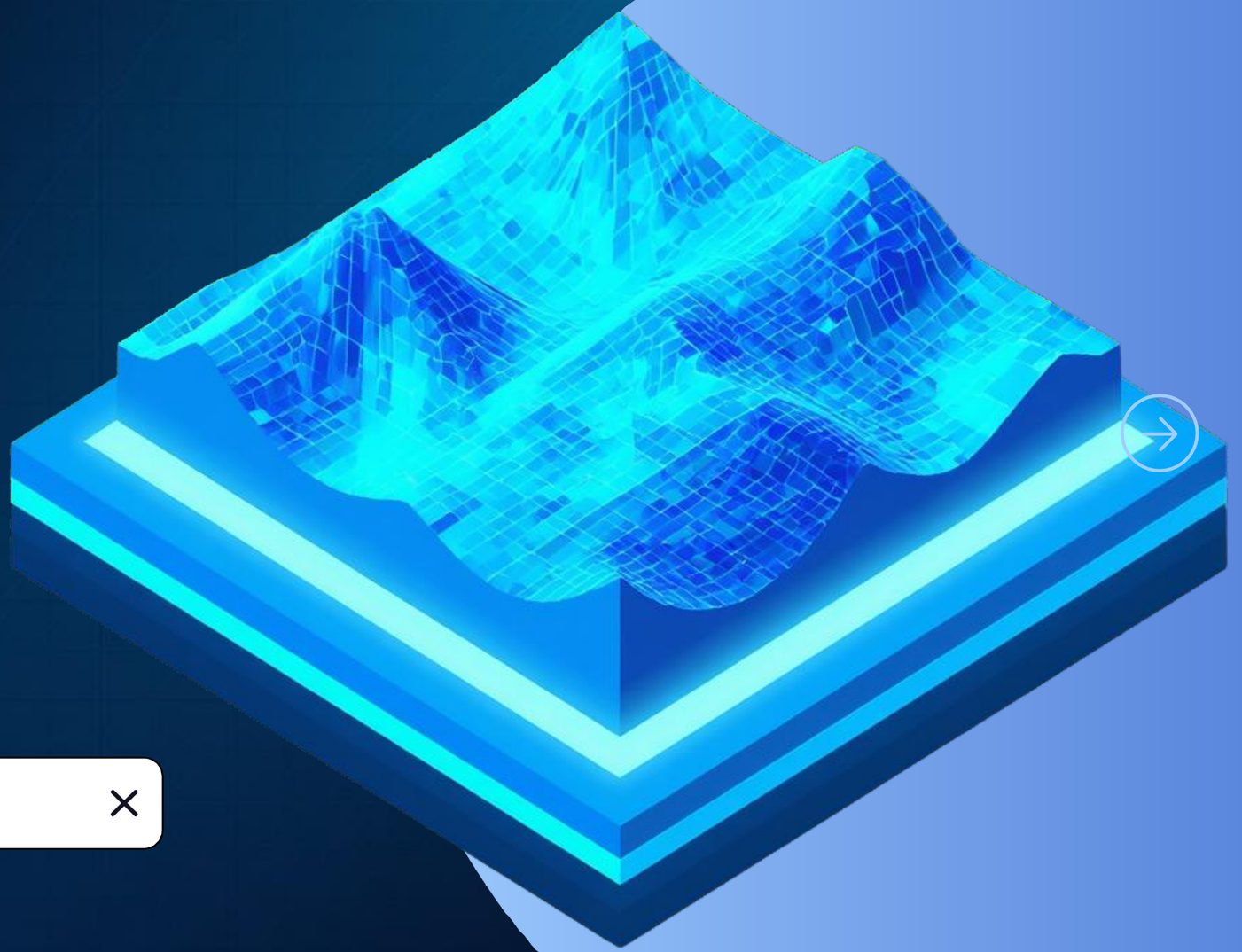
PYTHON FOR GEOSPATIAL

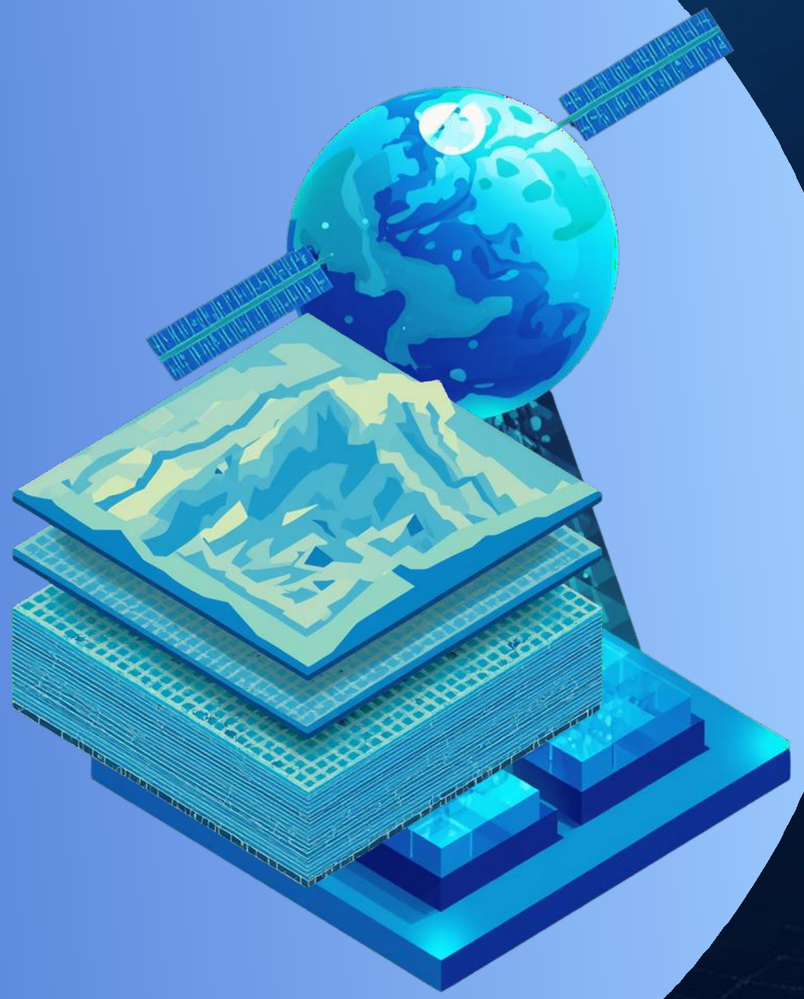
PART 2

ASEAN
GEOAI FUSION 2026

Duration: 30 minutes

Rasterio · Raster Processing · Folium · Jupyter



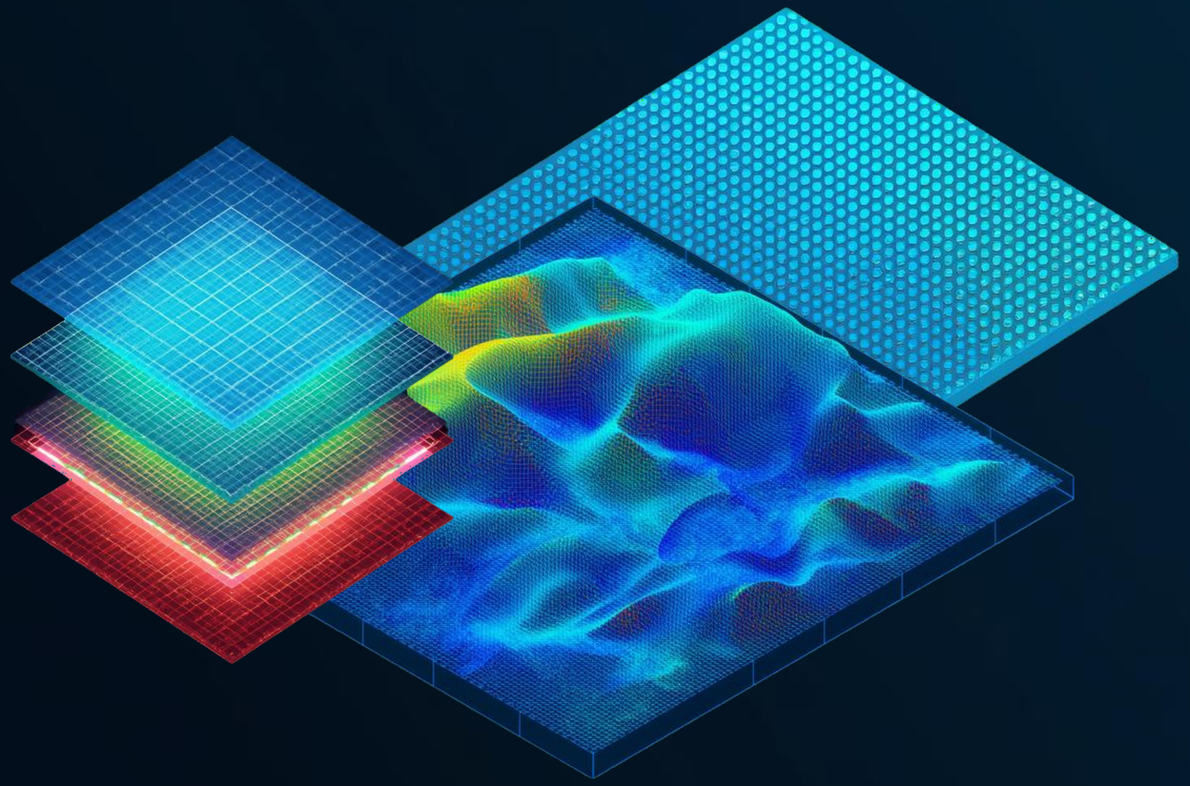


SESSION **OVERVIEW**

Learning Objectives

- Raster Data & Rasterio
- Folium & Web Mapping
- Jupyter Workflows

Part 2: Rasterio, Raster Processing, Folium & Jupyter



UNDERSTANDING RASTER DATA

Raster data represents the world as a continuous grid of cells (pixels), where each cell holds a value such as elevation, temperature, or reflectance.

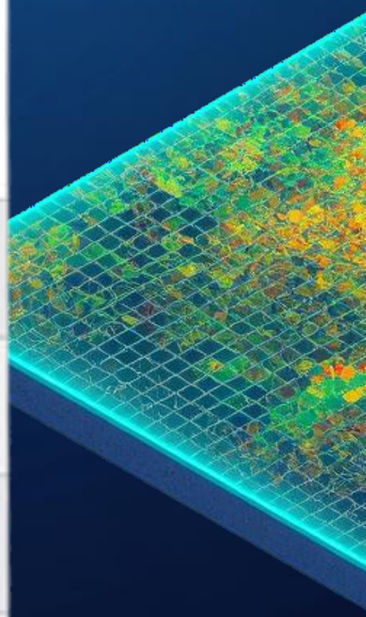
Unlike vector data, rasters divide space into uniform rows and columns – forming a pixel grid. Each pixel stores one or more band values. Common examples include satellite imagery (RGB/multispectral), Digital Elevation Models (DEMs), land cover classification maps, and NDVI composites.

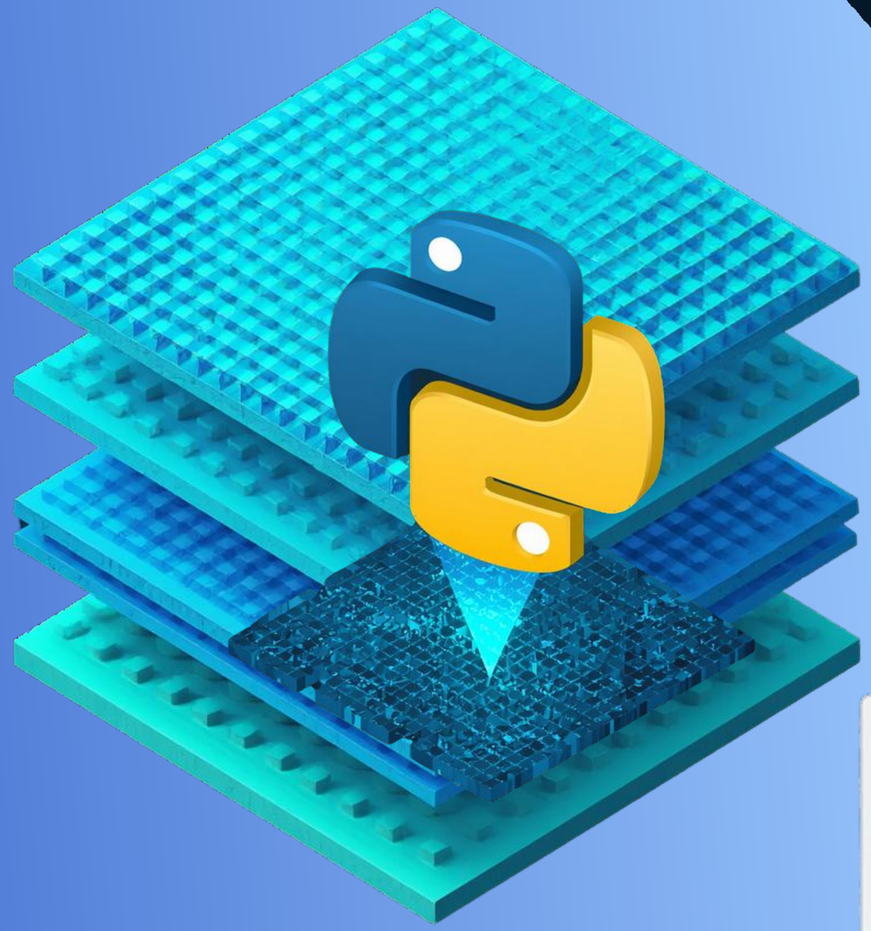
Raster structure consists of: Grid dimensions (rows × columns), Spatial resolution (cell size in map units), Coordinate Reference System (CRS), Band count (single or multi-band), and NoData values for missing areas. Understanding this structure is essential before processing rasters with Python tools like Rasterio.

Part 2: Rasterio, Raster Processing, Folium & Jupyter

VECTOR VS RASTER

	Vector	Raster
Representation	Points, Lines, Polygons	Pixels
Precision	High	Resolution dependent
Storage	Efficient	Large files
Best For	Boundaries, Roads	Imagery, Terrain





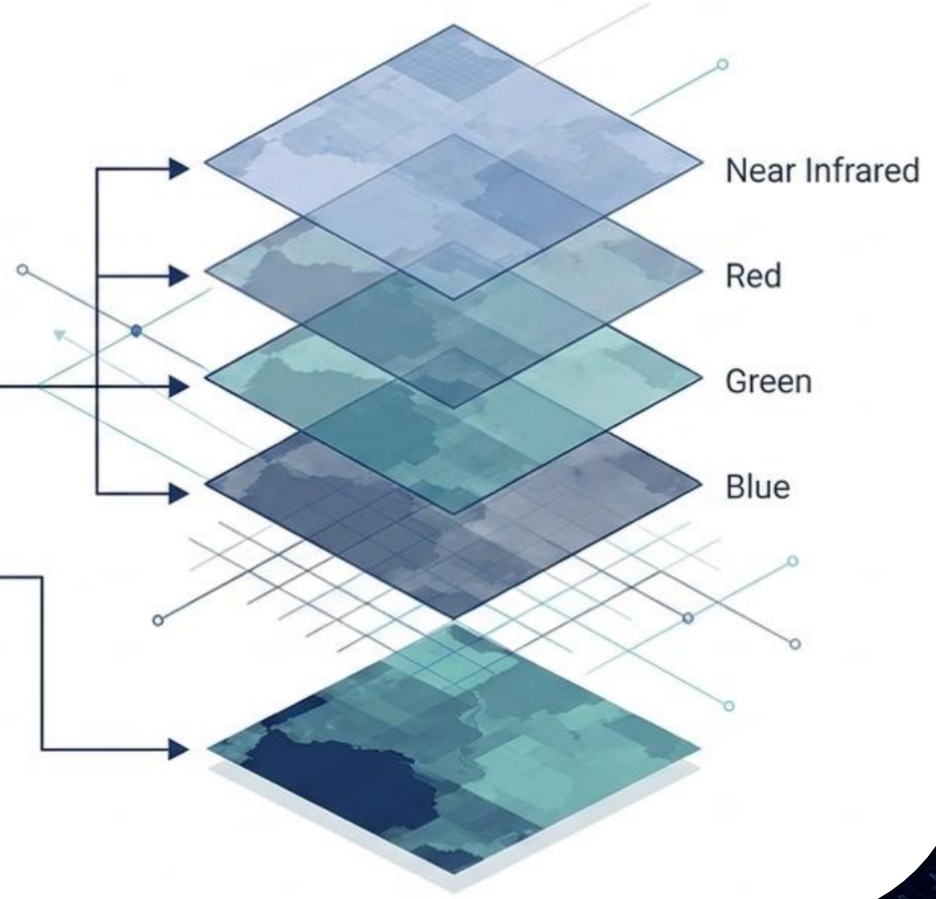
The processing engine: RASTERIO

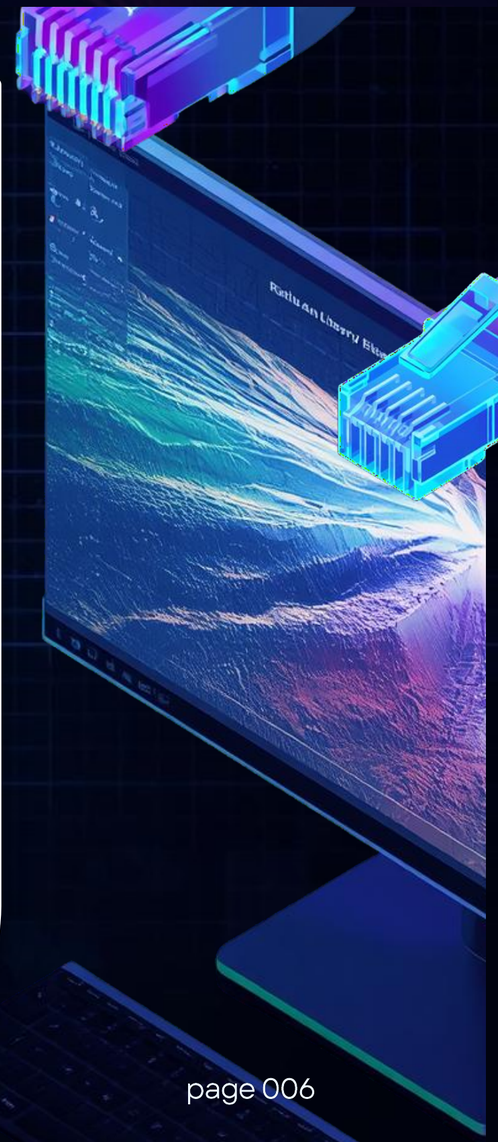
Rasterio is a powerful Python library for reading, writing and processing geospatial raster data.

Installation:
`pip install rasterio`

READING RASTER FILES

```
import rasterio

dataset = rasterio.open('landsat.tif')

dataset.width, dataset.height, dataset.count, dataset.crs


import matplotlib.pyplot as plt

plt.imshow(dataset.read(1))
```

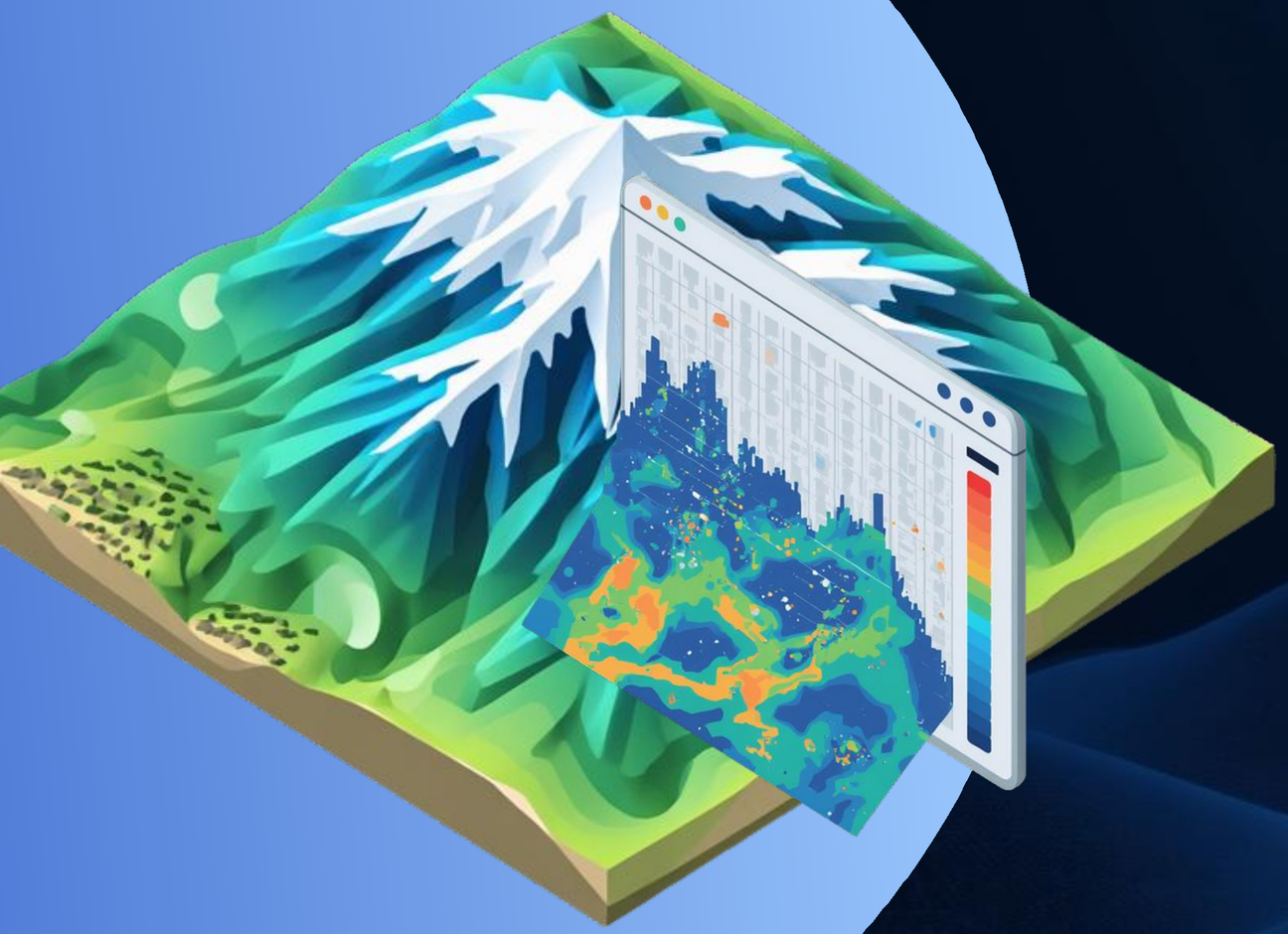




RASTER **METADATA**

- CRS & Projection
- Resolution
- Spatial Extent
- Bands & Spectra

Every raster file carries rich metadata that defines how it relates to the real world. The Coordinate Reference System (CRS) defines the spatial projection and datum, ensuring correct geographic placement. Spatial extent describes the bounding box – the minimum and maximum x/y coordinates covered by the raster. Resolution (pixel size) indicates the ground distance each pixel represents, directly affecting detail and file size. Bands encode spectral information: a single-band raster may represent elevation or temperature, while multi-band rasters (e.g., Landsat, Sentinel) store wavelengths across visible, near-infrared and shortwave-infrared channels. In Rasterio, all metadata is accessible via dataset.crs, dataset.transform, dataset.count, dataset.width, and dataset.height – enabling precise analysis and processing pipelines.



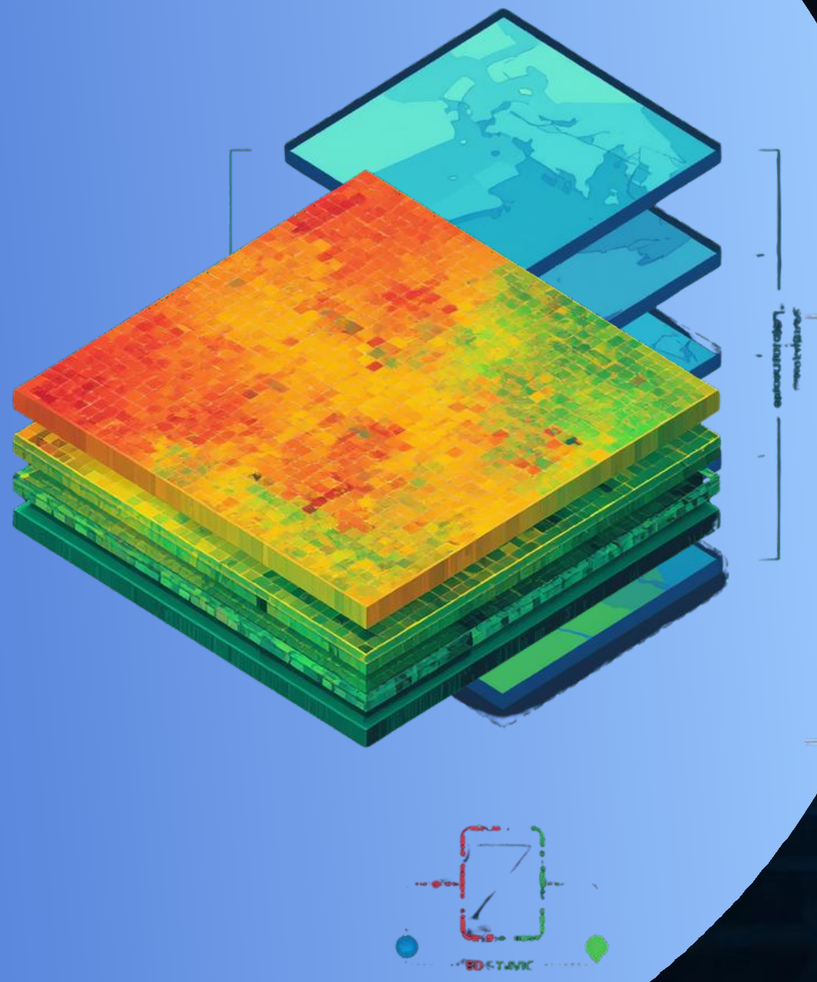
VISUALIZING RASTER DATA

Matplotlib Display: Use imshow() with colormaps (e.g., gray, terrain, RdYlGn) to render raster bands. Set extent from transform for accurate spatial axes.

Imagery Visualization: Read RGB bands from GeoTIFF, stack with np.dstack(), normalize to 0–1 range. Display true-color or false-color composites using rasterio.plot.show().

DEM Visualization: Render Digital Elevation Models using hillshade algorithms. Apply terrain colormap to highlight elevation gradients and topographic relief across study areas.

Key Functions: rasterio.plot.show(src), plt.imshow(band, cmap='terrain'), show_hist(src) for band histograms, and earthpy.plot for multi-band layouts.



RASTER **PROCESSING**

Clipping

Extracting specific areas of interest.

Resampling

Altering pixel resolution.

Reprojection

Transforming the Coordinate Reference System (CRS).

Band Math

Computing spatial indices.

Example: NDVI = (NIR - RED) / (NIR + RED)

FOLIUM

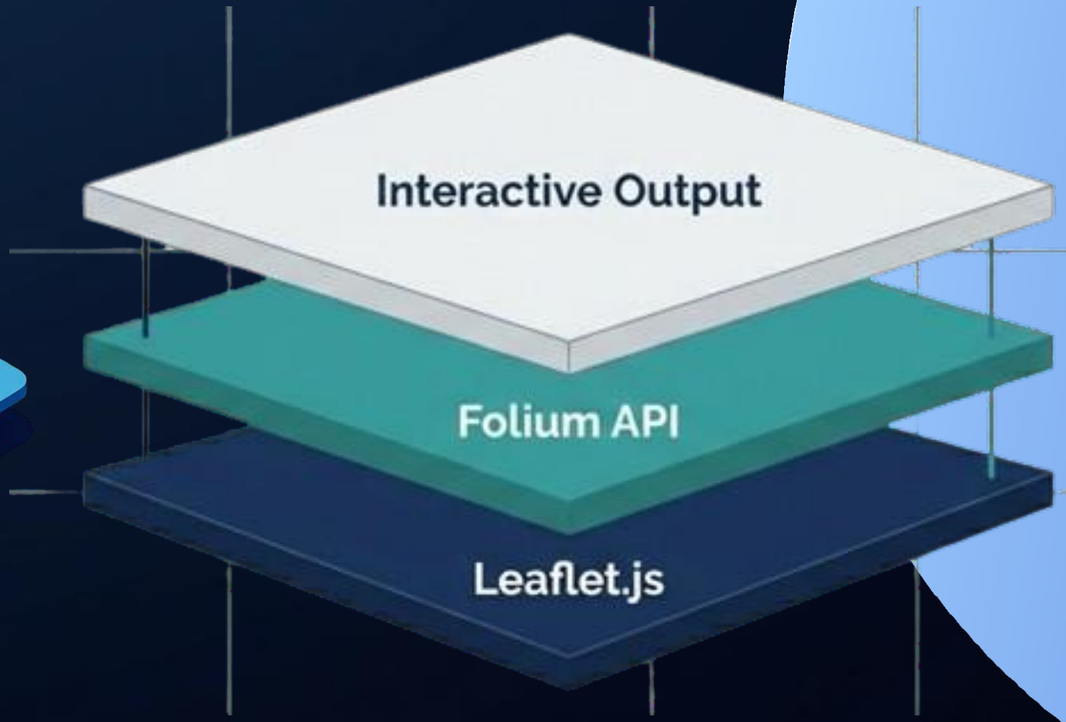
```
pip install folium
```

INTERACTIVE MAPS

Folium is a powerful Python library that enables the creation of interactive web maps using the Leaflet.js JavaScript library as its foundation.

- Leaflet Foundation: Renders maps in the browser using Leaflet.js – no JavaScript required from the user
- Easy Integration: Works seamlessly within Jupyter Notebooks and exports to standalone HTML
- Tile Layers: Supports OpenStreetMap, Stamen, CartoDB, and custom tile providers
- GeoJSON Support: Overlay vector data including points, lines, and polygons
- Choropleth Maps: Visualize spatial statistics with color-coded region fills
- Markers & Popups: Add interactive annotations and tooltips to map features
- Layer Control: Toggle multiple data layers on and off interactively.

FOLIUM
ARCHITECTURE STACK



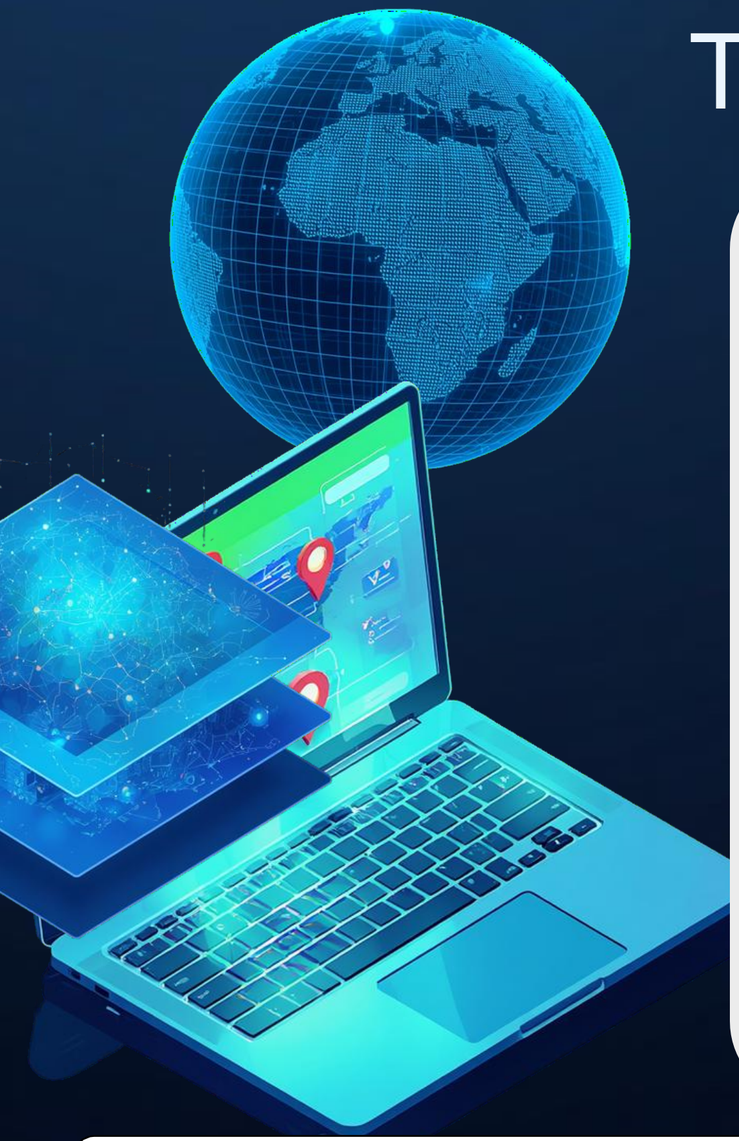
Key Features

- Zooming capabilities
- Dynamic Layer Control
- Interactive Popups & Markers
- Choropleth mapping

EXAMPLE

```
import folium

m = folium.Map(location=
[3.139,101.686], zoom_start=10)

folium.Marker([3.139,101.686],
popup='Kuala Lumpur').add_to(m)

folium.GeoJson(districts).add_t
to(m)
```

THE UNIFIED ECOSYSTEM BY PYTHON



The Reproducible Environment

- Interactive coding
- Inline visualization
- Integrated documentation
- End-to-end reproducibility

KEY TAKEAWAYS

In this session, we explored the full raster geospatial workflow using Python. From loading and inspecting GeoTIFF files with Rasterio, to processing and visualizing raster data, you gained hands-on experience with real-world geospatial datasets. We also introduced Folium for interactive web mapping and demonstrated how Jupyter Notebooks support reproducible, shareable GIS analysis workflows.

- Rasterio & Raster Processing
- Folium & Interactive Mapping
- Jupyter Workflow & Reproducibility